StaLLM: Benchmarking Prompts and LLMs in Software Maintenance

Tewfik Ziadi, Seifeddine Bouallegue, and Reda Bendraou
University of Doha for Science and Technology, Doha, Qatar
{tewfik.ziadi, seifeddine.bouallegue, reda.bendraou}@udst.edu.qa

Keywords: Large Language Models, Prompt Engineering, Software Maintenance, Static Analysis, Benchmarking

Abstract: Large Language Models (LLMs) are increasingly used in software maintenance and code-quality analysis, yet their outputs remain highly sensitive to prompt design, model choice, and evaluation setup, complicating systematic comparison. This is especially challenging for maintenance tasks that require localized, typed findings rather than non-structured responses. We present StaLLM, a benchmark for evaluating LLM-based code-quality analysis under a controlled and reproducible protocol. StaLLM combines (i) a multi-language benchmark built from real systems and static-analysis outputs, (ii) a modular prompt-execution pipeline, and (iii) an evaluation engine that normalizes LLM outputs and compares them against reference findings using span-level metrics. We illustrate the benchmark through two studies: prompt sensitivity (three prompt styles, fixed model) and model sensitivity (four LLM slots, fixed prompt). We also demonstrate extensibility using vulnerability findings. Results highlight clear differences in precision, recall, and coverage across prompts and models, underscoring the need for structured benchmarking in maintenance-oriented LLM usage.

1 Introduction

Large Language Models (LLMs) are increasingly used in software engineering to support tasks such as code completion, documentation generation, defect localization, quality analysis, and test generation (Jiang et al., 2023; Xia et al., 2023; Haque, 2025). Their strong performance on code-related tasks suggests significant potential for software maintenance, where developers must inspect large codebases, identify quality issues, and reason about corrective actions (Livshits and Lam, 2005; Ayewah et al., 2008). In practice, however, maintenance tasks differ from standard code-generation settings: they often involve partial evidence, large inputs, multiple plausible outputs, and strong dependence on contextual information.

This makes evaluation particularly difficult in maintenance settings. LLM behavior is sensitive to prompt formulation, input structure, and contextual cues, which complicates systematic comparison and reproducibility (Wu et al., 2022; Liu et al., 2024). In addition, many existing benchmarks focus on downstream programming tasks with relatively well-defined objectives, whereas maintenance-oriented scenarios require localized and typed findings that can be compared against reference analyses. Typical examples include identifying relevant files, recovering issue spans, and aligning model outputs with static-analysis results.

To address this gap, we introduce StaLLM, a benchmark for studying how prompt and model choices affect LLM-based code-quality analysis under a fixed evaluation protocol. StaLLM brings together (i) curated multi-language systems (Java, C#, PHP) enriched with static-analysis results, (ii) a modular prompt-execution pipeline, and (iii) an evaluation engine that normalizes LLM outputs and compares them against grounded reference findings using span-level, maintenance-oriented metrics. A key design choice is provider-agnostic configuration through environment "slots", enabling reproducible experiments across Azure OpenAI, OpenAI, and local Ollama deployments without code changes. Rather than focusing only on correctness in closed tasks, StaLLM targets maintenance-oriented analysis under practical constraints such as Top-k file selection, large codebases, limited context windows, and prompt verbosity control. The contributions of this paper are as follows:

- We introduce StaLLM, an open-source benchmark for comparing prompt-engineering strategies in LLM-based software maintenance, with provider-agnostic configuration for reproducible experimentation.
- We provide a reusable multi-language benchmark based on real systems, ArgoUML (Java), eSho-

pOnContainers (C#), and Magento (PHP), enriched with static-analysis results and integrated into a unified prompt-execution and evaluation pipeline.

- We implement an automated evaluation engine that normalizes and compares LLM outputs against reference findings using maintenance-oriented metrics, supporting large-scale, Top-k-driven experiments.
- We report a concrete use of STALLM through three representative prompt strategies and four LLMs, showing how prompt style and model choice affect issue detection, coverage, redundancy, and alignment with static-analysis findings.

The remainder of this paper is organized as follows. Section 2 motivates the problem and reviews background on code-quality analysis and LLMs. Section 3 presents the STALLM benchmark, its design, and its main components. Section 4 illustrates its use through two studies (E1: prompt sensitivity, E2: model sensitivity). Section 5 surveys related work. Finally, Section 6 concludes the paper and outlines future directions.

2 Background and Motivations

Static code analysis supports several maintenance activities, including defect prevention, security assessment, refactoring, and technical-debt management. A static analysis engine processes source files without executing them, applies predefined rules and metrics, and produces findings such as code smells, bugs, and vulnerabilities. These findings are typed, prioritized, and localized to precise code positions (file, line, column), which makes them suitable as grounded reference outputs for maintenance-oriented evaluation. Figure 1 summarizes this general workflow.

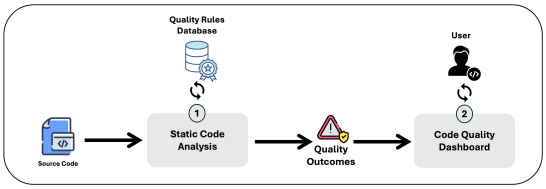


Figure 1: Overall static analysis workflow: source code → analysis engine (driven by rules/metrics) → quality outcomes → dashboard (Ghallab et al., 2024).

Numerous tools operationalize this process; SonarQube is a prominent example (Vassallo et al., 2020). Figure 2 shows a typical quality outcome produced by SonarQube, including the issue type and its source-code location. More broadly, quality-oriented static analysis supports multiple maintenance goals, including maintainability, defect prevention, and security (dos Reis et al., 2021; Azeem et al., 2019; Zakeri-Nasrabadi et al., 2023).

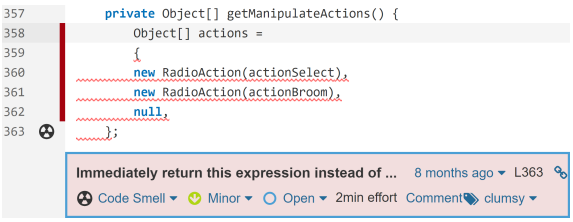


Figure 2: Example of quality outcome in SonarQube (Ghallab et al., 2024).

2.1 LLMs for Code Quality Analysis

Recent work positions Large Language Models (LLMs) as complementary analyzers for code quality and security. Surveys synthesize their capabilities and limitations across software-engineering tasks (Hou et al., 2024), while empirical studies investigate specific goals such as vulnerability detection (Lin and Mohaisen, 2025) and code-smell identification (Silva et al., 2024). These results are promising, but they also highlight the need for structured and reproducible evaluation protocols.

In practice, using LLMs for code-quality analysis typically involves three steps: (i) prompt design, which specifies the analysis goal, target language, and expected output schema; (ii) model selection with fixed inference settings; and (iii) structured result extraction in a machine-readable format that enables automated comparison with reference findings from static analyzers. The following prompt illustrates this principle with a compact, schema-constrained query.

Prompt Example

You are a code-quality analyzer for Java. Detect in the source code below high-confidence issues. Return a JSON array only; each item has "type","description", and either "line" or ("startLine","endLine") as integers (max 5). Return [] if none. Code: {code}.

2.2 Motivations

Despite recent progress, systematic evaluation of LLMs for maintenance-oriented analysis remains difficult. Existing benchmarks increasingly improve experimental control (Dubniczky et al., 2025; Lin and Mohaisen, 2025), but several practical obstacles remain. First, *evaluation granularity* is challenging: maintenance findings are typed and localized to code

spans, whereas many benchmarks emphasize generation or whole-file judgments. Exact matching is brittle under small localization shifts, which motivates tolerant span-level matching. Second, *realistic sampling* matters: real projects contain both positive and negative files, so evaluation should make the search universe and Top-k budget explicit rather than focusing only on positive cases. Third, *reproducibility and comparability* remain difficult because LLM outputs depend on prompt wording, verbosity constraints, inference settings, context windows, and provider deployments (Hou et al., 2024). Finally, *ground-truth heterogeneity* complicates comparison: static analyzers provide typed and localized findings, but rule naming and span granularity vary across tools, which motivates normalization through adapters and unified schemas (Vassallo et al., 2020).

STALLM addresses these needs through a lightweight benchmarking protocol that enforces schema-constrained outputs, uses tolerant span-level scoring, makes the search budget explicit, and records experimental factors such as prompt style, matching strictness, model choice, and execution cost. The next section presents the benchmark and pipeline in detail.

3 STALLM: A Benchmark for LLM-Based Code Quality Analysis

STALLM provides a reproducible benchmark for evaluating prompt strategies and LLM choices in code-quality analysis. It operates on datasets with pre-existing ground truth (GT) quality outcomes and enforces normalized outputs (e.g., strict JSON), enabling direct comparison with practitioner baselines. Figure 3 overviews the benchmark workflow.

3.1 Main Components

Dataset & Ground Truth. STALLM integrates three realistic case studies drawn from the literature (Ghallab et al., 2024): ArgoUML (Java), eShopOnContainers (C#), and Magento (PHP). The selection is motivated by language diversity and by the availability of high-quality static-analysis results produced by SonarQube. For each system, the GT contains issue categories and precise locations, enabling direct comparison between LLM findings and scanner outputs. Table 1 summarizes the corpus. Each GT item is stored as {`file`, `line`/(`startLine`, `endLine`), `column`/(`startColumn`, `endColumn`), `type`}, where `type` denotes the issue category (smell/defect/security).

Sampling Configuration (Top-k & Positive Ratio). STALLM exposes a *Top-k files* parameter that controls the evaluation universe U, i.e., the set of files sent to the LLM and used for span-level scoring. Users configure (i) the total number of files in U, (ii) the target fraction of positive files, and (iii) the matching criterion used to compare LLM findings with GT, ranging from coarse type-level agreement to strict span matching. By default, STALLM fills U by selecting positive files first, prioritizing files with more GT items, and then sampling negative files uniformly at random to reach k.[1] This makes the search budget explicit and ensures reproducible runs.

Prompt & LLM Repositories. STALLM is designed to assess how both *prompt choice* and *LLM choice* influence code-quality analysis. It therefore maintains a repository of predefined prompts that users can execute directly or extend for side-by-side comparison. It also maintains a registry of models identified by simple backend descriptors (e.g., `openai:gpt-4o`, `azure:gpt-4o-mini@deploymentX`, `ollama:codellama`). API keys and local endpoints remain external to the benchmark, while each run records the model identifier, version, and decoding policy for reproducibility.

Metrics Generator. STALLM compares LLM findings against GT quality outcomes for the selected system. Each finding specifies an issue type and a location in the source code (single line or span). A matching finding is counted as a true positive (TP), a non-matching finding as a false positive (FP), and any unmatched GT item as a false negative (FN). From these counts, STALLM reports standard effectiveness metrics, namely precision, recall, and F_1, through the dashboard.

3.2 Implementation

STALLM is implemented as a web application composed of a dataset and GT manager, an experiment designer, an execution engine for LLM calls, and a results dashboard. Figure 4 shows the main screen, where the user selects the task, uploads the GT, configures the input corpus, and chooses the evaluation settings. STALLM supports complementary comparison settings along two orthogonal axes: prompts and models. Users can run a single prompt or multiple prompts, a single LLM or multiple LLMs, or

[1] Negative files help quantify false positives on clean code.

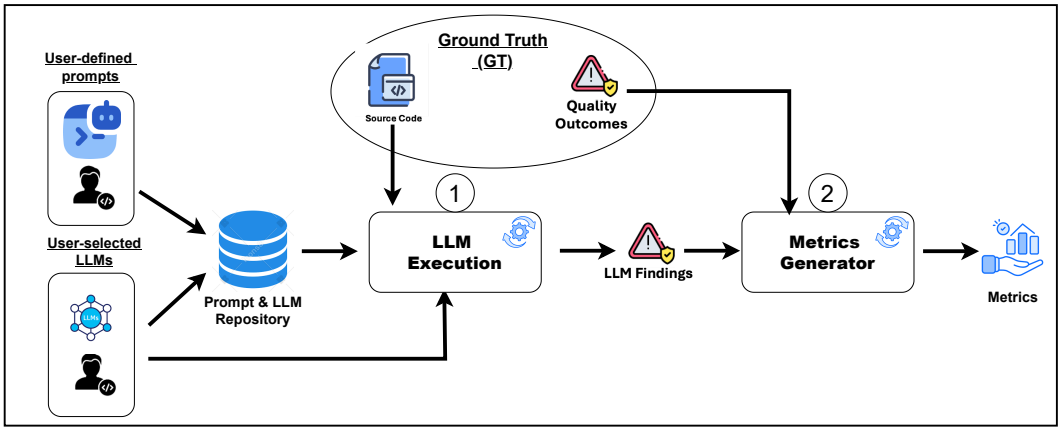


Figure 3: StALLM workflow. User-defined prompts and selected LLMs are stored in repositories, combined during (1) LLM execution to produce normalized findings, and evaluated against pre-existing ground truth (code + quality outcomes) in (2) the metrics generator.

Table 1: Case studies and ground truth (LOC, language, number of quality outcomes, and repository).

Case Study	Language	LOC	# Quality Outcomes	Repository
ArgoUML	Java	459,391	12,345	github.com/argouml-tigris/argouml
eShopOnContainers	C#	87,542	820	github.com/dotnet-architecture/eShopOnContainers
Magento	PHP	4,147,711	24,225	github.com/magento/magento2

the full grid (multiple prompts $\times$ multiple models) in a controlled manner. The engine batches requests, records inputs and outputs for reproducibility, and computes the selected metrics before presenting interactive summaries on the dashboard. The framework is available as an open-source project on GitHub[2].

The next section illustrates the use of StALLM through two concrete experiments.

4 Example of StALLM Usage

This section illustrates how StALLM can be used in practice through two lightweight experiments: (i) a *prompt-sensitivity* study with a fixed LLM, and (ii) a *model-sensitivity* study with a fixed prompt. We intentionally keep the setup simple and compact.

Concretely, we evaluate on the eShopOnContainers codebase (C#) using a ground truth (GT) CSV produced by running SonarQube on the entire project. For each run, we sample the file universe U by selecting a Top-k budget and targeting a *positive ratio* (the share of files containing at least one GT span). StALLM then matches predicted spans to GT spans *within the same file* using a tolerant rule: optional type agreement together with either IoU $\geq \tau$ or line distance $\leq \delta$; for single-line spans with columns, the same criterion applies to columns. We report span-level precision, recall, and F1, together with latency and token/cost information.

4.1 Prompt Variants and LLMs

Prompt variants. We evaluate three concise prompt styles, shown below, that target different trade-offs between coverage and precision. All prompts return one issue per line using the pattern `[type] @ line(s): short description`. The *Baseline* prompt is precision-oriented: it asks for only a few high-confidence findings, thereby limiting hallucinations and review effort. The *Scanner* prompt is coverage-oriented: it encourages the model to surface all meaningful issues (style, maintainability, complexity), accepting a higher false-positive rate. The *Hybrid* prompt follows a two-phase policy, broad search followed by self-filtering, to produce a mid-sized but cleaner result set.

[2]https://github.com/tziadi/StaLLM

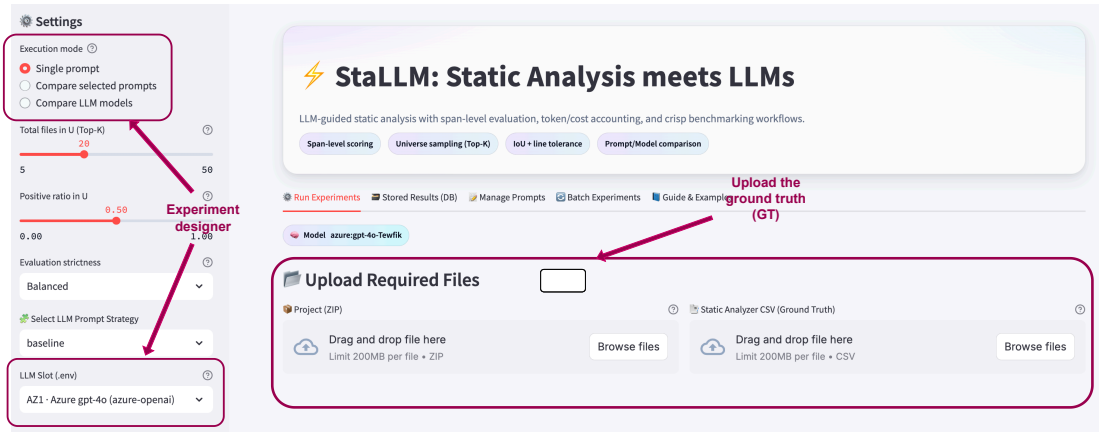


Figure 4: Main screen of STALLM.

Baseline Prompt (concise)

You are a software quality analyzer for {language}. Report only high-confidence maintainability issues (no speculation), max 5.
Output format (one item per line):
`[type] @ line(s): short description`
Language: {language}
Code: {code}

Scanner Prompt (broad)

You act like a static analyzer (e.g., SonarQube) for {language}. Report all meaningful issues (style, maintainability, complexity, basic correctness), up to 15. De-duplicate near-identical items.
Output format (one item per line):
`[type] @ line(s): short description`
Language: {language}
Code: {code}

Hybrid Prompt (balanced)

You are both a static analyzer and a careful human auditor for {language}. Find issues broadly, then self-filter to keep 5–8 clear, actionable items (balancing coverage and precision). Merge near-duplicates.
Output format (one item per line):
`[type] @ line(s): short description`
Language: {language}
Code: {code}

LLMs used. We configure four slots in STALLM and switch models through configuration only: AZ1 (hosted API, `gpt-4o`), AZ2 (hosted API, `gpt-4.1`), AZ3 (hosted API, `gpt-o3`), and AZ4 (*local* runtime via Ollama, `deepseek-coder:6.7b-instruct`, unless stated otherwise). To keep the discussion vendor-neutral, we refer to slots rather than providers. In Experiment E1, we fix the LLM to AZ1 to isolate prompt sensitivity; in Experiment E2, we fix the prompt and compare AZ1–AZ4 to study model sensitivity. All runs share the same output schema and span-level scoring; only the active slot changes.

Evaluation in one line. STALLM matches predicted line spans to ground truth (GT) spans *within the same file* using a tolerant rule (IoU and a small line tolerance δ), then reports span-level precision, recall, and F1.

4.2 Experiment E1: Prompt Sensitivity

Setup. We run STALLM on the eShopOnContainers codebase (C#) with a fixed LLM (slot AZ1: Azure `gpt-4o`). For this experiment, we sample U with Top-$k = 20$ files and a positive ratio of 1.00; the preset is *Lenient*.

We compare three concise prompts: (i) *Baseline* (precision-focused, max 5 items), (ii) *Scanner* (broader coverage, up to 15, de-duplicated), and (iii) *Hybrid* (self-filtering to 5–8 items). All prompts emit one issue per line with an issue type and a line span.

Results. Table 2 reports span-level precision, recall, F1, and wall-clock time. *Scanner* achieves the highest recall and F1, *Hybrid* yields the best precision and the shortest runtime, and *Baseline* lies in between.

Prompt	Precision	Recall	F1	Time (s)
Baseline	0.361	0.272	0.310	61.9
Scanner	0.307	0.441	0.362	100.4
Hybrid	0.425	0.262	0.324	57.8

Table 2: E1 (eShop, AZ1=`gpt-4o`, Top-$k = 20$, positive ratio = 1.00, *Lenient*): prompt sensitivity with latency.

Discussion. For quick triage under tight latency constraints, *Hybrid* provides the strongest precision and runs fastest. When coverage matters most, *Scanner* substantially improves recall and yields the best F1, at the cost of higher runtime. *Baseline* remains a reasonable middle ground, with moderate latency and moderate precision.

4.3 Experiment E2: Model Sensitivity

Setup. We keep the best-performing prompt from E1 (*Scanner*) and vary the model by switching STALLM slots: AZ1=`gpt-4o`, AZ2=`gpt-4.1`, AZ3=`gpt-o3`, and AZ4=`deepseek-coder:6.7b-instruct` running locally via Ollama. We evaluate on the same eShopOnContainers subset (C#), with Top-$k = 20$, positive ratio = 1.00, and the *Lenient* preset ($\tau = 0.10$, $\delta = 3$).

Results. Table 3 reports span-level precision, recall, F1, and wall-clock time. The hosted GPT-class models (AZ1–AZ3) achieve the strongest F1 values, with `gpt-4o` and `gpt-o3` mainly trading precision against recall. The local `deepseek-coder` model (AZ4) obtains substantially lower F1 in our setting.

Model (slot)	Precision	Recall	F1	Time (s)
GPT-4o (AZ1)	0.38	0.28	0.32	69
GPT-4.1 (AZ2)	0.36	0.30	0.33	72
GPT-o3 (AZ3)	0.34	0.34	0.34	77
deepseek-coder 6.7B (AZ4, local)	0.18	0.10	0.13	41

Table 3: E2 (eShop, fixed prompt = *Scanner*, Top-$k = 20$, positive ratio = 1.00, *Lenient*): model sensitivity across hosted GPT-class models and a local Ollama model.

Discussion. (1) *Hosted vs. local.* The local 6.7B code model (AZ4) is attractive for privacy and cost, but lags in span-level F1 on this task and dataset; in our runs, it produced more off-target spans and more under-reporting. (2) *GPT-class trade-offs.* `gpt-4o` tends to provide higher precision, whereas `gpt-o3` improves recall, yielding comparable F1 values.

4.4 Extensibility

STALLM is agnostic to the ground-truth (GT) source and supports lightweight adapters that map external formats to the internal span schema `{file, start, end, type, meta}`. We also ingest vulnerability findings from *Snyk* (Snyk, 2025b; Snyk, 2025a) (free tier) as CSV (`file`, `rule_id`/`CWE`, `severity`, `line_start`, `line_end`) and evaluate a dedicated vulnerability prompt without modifying the engine.

Vulnerability Prompt (security)

You are an application security reviewer for {language}. Report up to 10 credible vulnerabilities (no speculation), preferring precise spans.
Output: `[type/CWE|severity] @ line(s): short rationale`
Language: {language} Code: {code}

Results and discussion. Under the same setting as E1 (Top-$k = 20$, *Lenient*), the vulnerability task achieved Precision 6.06%, Recall 12.90%, and F1 8.25% (time ≈ 39.5 s). These lower scores reflect (i) the sparsity of security findings, (ii) mismatches between prompt and scanner rules, and (iii) limitations of the *free* Snyk tier. Stronger prompts, larger Top-k, and richer GT sources are expected to improve recall and alignment.

Extensible ground truth. STALLM supports GT from static-analysis tools (e.g., SonarQube) and vulnerability scanners via lightweight adapters (e.g., CSV $\rightarrow$ {*file, start, end, type, meta*}). This preserves span-level evaluation while enabling vulnerability-oriented prompts and per-type analyses.

5 Related Work

Static analysis and quality assessment. Static code analysis supports maintenance activities such as defect prevention, security assessment, refactoring, and technical-debt management (see Section 2). Tools such as SonarQube operationalize rule-based assessment at scale, producing typed findings with precise locations and severities (Vassallo et al., 2020). More broadly, the literature consolidates design- and code-level indicators through surveys on smell detection and visualization (dos Reis et al., 2021), machine-learning approaches for smell identification (Azeem et al., 2019), and datasets and validation protocols for evaluating detectors (Zakeri-Nasrabadi et al., 2023). Because these outputs are typed and localized to files and line spans, they provide a practical and auditable reference for downstream comparison.

LLMs for code quality and security. Large Language Models (LLMs) have been studied extensively

across software-engineering tasks (Hou et al., 2024), with more targeted work on vulnerability detection under controlled settings (Lin and Mohaisen, 2025) and on code smells as a quality signal (Silva et al., 2024). These studies suggest that LLMs can complement traditional analyzers, but they also show that results depend strongly on prompt formulation, input structure, and context limits, which complicates reproducibility and fair comparison (Wu et al., 2022; Liu et al., 2024).

Benchmarks and their focus. Existing community benchmarks mostly target tasks with clear success criteria, such as code understanding and generation (CodeXGLUE (Lu et al., 2021)) or repository-level issue resolution (SWE-bench (Jimenez et al., 2024)). More recent efforts begin to connect static analysis and LLMs; for example, CASTLE evaluates analyzers and LLMs on vulnerability detection under controlled conditions (Dubniczky et al., 2025). However, these benchmarks do not directly address the setting targeted in this paper, where outputs are typed, localized to code spans, and compared against heterogeneous maintenance findings extracted from static-analysis tools.

Positioning of STALLM. STALLM is not intended to replace static analyzers; instead, it uses their outputs as grounded references to assess LLM behavior on maintenance-oriented analysis tasks. Concretely, STALLM (i) enforces a schema-constrained output of typed issues with line spans, (ii) evaluates predictions at the span level using a tolerant matcher (IoU with small line jitter), and (iii) makes experimental factors explicit and reproducible, including prompt style, Top-k sampling, positive ratio, matching strictness, and model choice, through a provider-agnostic, slot-based configuration (Azure OpenAI, OpenAI, local Ollama). In this sense, STALLM complements synthesis-oriented metrics and differs from multi-task or repair-focused suites by evaluating whether LLMs can recover maintenance findings under explicit span-level and budget constraints (Austin et al., 2021; Lu et al., 2021; Jimenez et al., 2024; Dubniczky et al., 2025).

6 Conclusion and Perspectives

We introduced STALLM, a lightweight, provider-agnostic benchmark for evaluating LLMs on maintenance-oriented code analysis with span-level scoring. By combining multi-language corpora, static-analysis ground truth, a slot-based execution setup (Azure/OpenAI/Ollama), and a unified protocol, STALLM enables controlled measurement of prompt and model sensitivity. Our study (E1/E2) shows that prompt design shifts the precision–recall balance under a fixed model, while model choice significantly impacts results under a fixed prompt. In our setting, local models offered advantages in cost and latency but lagged behind hosted models in accuracy.

We outline three directions: (i) extending the benchmark to tasks such as feature and bug localization with richer ground truth, (ii) enriching evaluation with ranking and calibration metrics and explicit quality, cost, and latency trade-offs, and (iii) studying long-context and retrieval-augmented approaches for larger codebases under controlled settings.

Results remain dependent on prompt design, dataset composition, and ground-truth noise. Although STALLM mitigates these factors through reproducible configurations, broader validation across systems is needed. Overall, maintenance-oriented LLM evaluation benefits from explicit ground truth, explicit budgets, and span-level scoring, and STALLM provides a practical benchmark to support this.

REFERENCES

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C., Heek, J., Agrawal, K., and Sohl-Dickstein, J. (2021). Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*. MBPP benchmark.

Ayewah, N., Hovemeyer, D., Morgenthaler, J. D., Penix, J., and Pugh, W. (2008). Using static analysis to find bugs. *IEEE Softw.*, 25(5):22–29.

Azeem, M. I., Palomba, F., Shi, L., and Wang, Q. (2019). Machine learning techniques for code smell detection: A systematic literature review and meta-analysis. *Information and Software Technology*, 108:115–138.

dos Reis, J. P., e Abreu, F. B., de Figueiredo Carneiro, G., and Anslow, C. (2021). Code smells detection and visualization: A systematic literature review. *Archives of Computational Methods in Engineering*, 28(8):5367–5396.

Dubniczky, R. A., Horvát, K. Z., Bisztray, T., Ferrag, M. A., Cordeiro, L. C., and Tihanyi, N. (2025). Castle: Benchmarking dataset for static code analyzers and llms towards cwe detection. In *Theoretical Aspects of Software Engineering: 19th International Symposium, TASE 2025, Limassol, Cyprus, July 14–16, 2025, Proceedings*, page 253–272, Berlin, Heidelberg. Springer-Verlag.

Ghallab, K., Ziadi, T., and Chalal, Z. (2024). An extensible feature-oriented approach for fine-grained code

quality analysis. In Chiba, S. and Thüm, T., editors, *Proceedings of the 23rd ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences, GPCE 2024, Pasadena, CA, USA, October 21-22, 2024*, pages 15–28. ACM.

Haque, M. A. (2025). Llms: A game-changer for software engineers? *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 5(1):100204.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., and Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 33(8):1–79.

Jiang, N., Liu, K., Lutellier, T., and Tan, L. (2023). Impact of code language models on automated program repair. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1430–1442. IEEE/ACM.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. (2024). SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Lin, J. and Mohaisen, D. (2025). From large to mammoth: A comparative evaluation of large language models in vulnerability detection. In *Network and Distributed System Security Symposium (NDSS)*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Livshits, B. and Lam, M. S. (2005). Finding security vulnerabilities in java applications with static analysis. In *USENIX Security Symposium*. USENIX Association.

Lu, S., Guo, D., Ren, S., Huang, J., et al. (2021). Codexglue: A machine learning benchmark dataset for code understanding and generation. In *NeurIPS 2021 Datasets and Benchmarks Track*.

Silva, L. L., Valente, M. T., Cedrim, D., and de Almeida Maia, M. (2024). Detecting code smells using chatgpt: Initial insights. In *Proc. of the ACM/IEEE Int'l Symp. on Empirical Software Engineering and Measurement (ESEM) — Emerging Results*. ACM.

Snyk (2025a). Snyk code: Static application security testing. https://snyk.io/product/code-security/. Accessed: 2025-10-17.

Snyk (2025b). Snyk: Developer security platform. https://snyk.io/. Accessed: 2025-10-17.

Vassallo, C., Panichella, S., Palomba, F., Proksch, S., Gall, H. C., and Zaidman, A. (2020). How developers engage with static analysis tools in different contexts. *Empir. Softw. Eng.*, 25(2):1419–1457.

Wu, T., Jiang, E., Donsbach, A., Gray, J., Molina, A., Terry, M., and Cai, C. J. (2022). Promptchainer: Chaining large language model prompts through visual programming. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM.

Xia, C. S., Wei, Y., and Zhang, L. (2023). Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 1482–1494. IEEE Press.

Zakeri-Nasrabadi, M., Parsa, S., Esmaili, E., and Palomba, F. (2023). A systematic literature review on the code smells datasets and validation mechanisms. *ACM Comput. Surv.*, 55(13s).